1. Executive Summary

This report documents the design, implementation and verification of **NER-Connect**, an AI-assisted logistics and road-accessibility intelligence platform for India's North Eastern Region, developed for the Ministry of Development of North Eastern Region (MDoNER) as a Smart India Hackathon prototype. The audit covers three delivered artefacts: a seventeen-route web application (React 18 + Vite + Tailwind CSS v4 + Leaflet), a FastAPI backend service with a risk-weighted route optimizer and OpenRouteService (ORS) integration, and a native Android application (Kotlin + Jetpack Compose) distributed as a signed APK.

17 / 17
web routes swept clean - zero blank pages, zero console errors

26 / 21
monitored NH segments and network nodes in the live graph

114 km
ORS-validated fastest road route rendered on the planner

15.6 MB
signed debug APK (V2 scheme) built with Gradle 8.14.3 + JDK 24

All three artefacts share one design language (the Civil Infrastructure and Logistics system: command-spruce primary, IMD 4-tier severity matrix, monospace telemetry) and one data story (26 road segments, 17 network nodes, 10 shipments, seeded alerts). The web and mobile clients run without authentication in the current demonstration configuration; the login, OTP and onboarding screens remain implemented but are bypassed in the default flow.

2. Scope and Deliverables

Three codebases were produced and audited. Each is independently runnable and shares dataset shape and design tokens with the others.

Deliverable	Stack	Entry point	Status
Web prototype	React 18.3 + Vite + Tailwind v4 + Leaflet 1.9	prototype/ - npm run dev (port 5174)	Verified, 17/17 routes
Backend service	Python 3.14 + FastAPI 0.115 + uvicorn	backend/ - uvicorn main:app (port 8000)	Verified, ORS live
Android app	Kotlin 2.1.20 + Jetpack Compose + AGP 8.13	android/apk/NER-Connect-v1.0-debug.apk	Built, V2-signed

3. System Architecture

The platform follows a four-layer architecture. Clients talk only to the FastAPI gateway; the gateway delegates to stateless engine modules; engines read the in-memory seed dataset (production target: PostGIS). External services are reached from exactly two places: the browser loads OSM raster tiles directly, and the server-side ORS client calls the OpenRouteService directions API. No API key ever reaches the browser bundle.

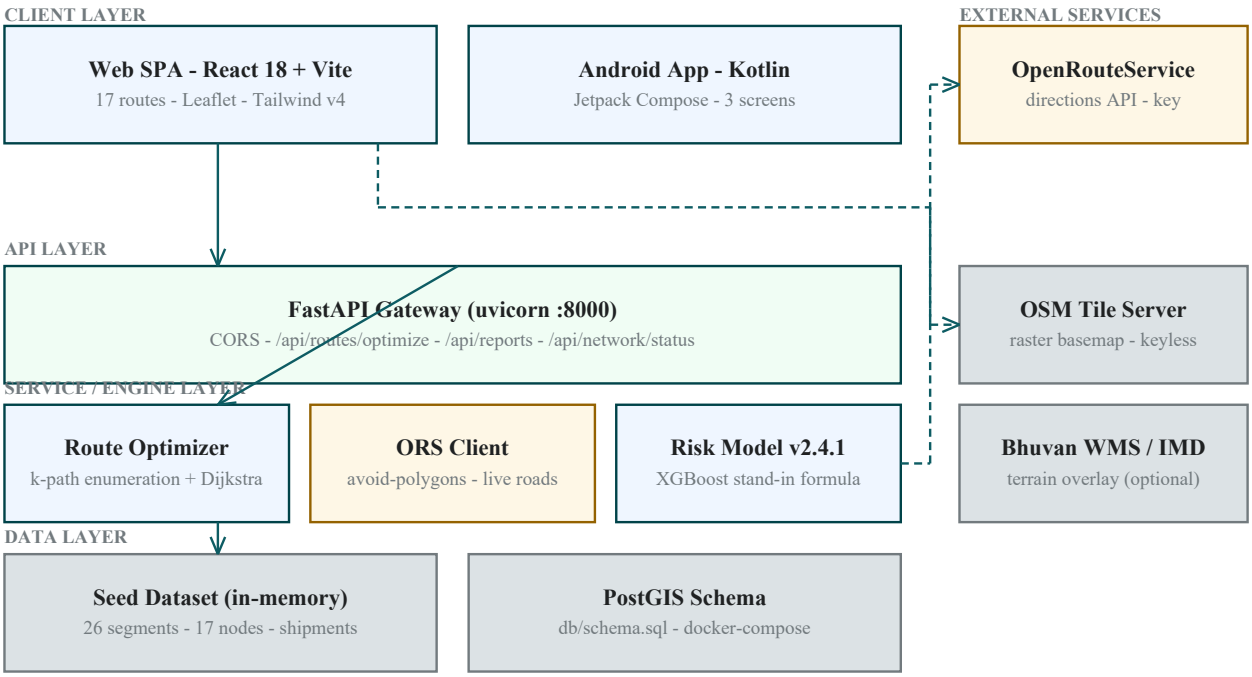


Figure 1 - NER-Connect layered architecture. Solid arrows are request/data flow; dashed arrows are external HTTP calls.

Two deliberate resilience decisions shape this architecture. First, the web client embeds a complete fallback engine (k-path enumeration over the same road graph with cargo-weighted risk), so every screen renders even when the backend is unreachable - the route planner silently reverts to the designed corridor. Second, the ORS integration degrades gracefully: if the key is absent, the quota is exhausted, or a route exceeds ORS limits, the response falls back to the graph engine result without user-visible failure.

4. Technology Stack Audit

Layer	Technology	Version	Assessment
Web framework	React + Vite + react-router-dom	18.3.1 / 5.4 / 6.x	Current LTS pair; React 19 blocked by react-leaflet 4.x peer range
Styling	Tailwind CSS v4 (@tailwindcss/vite)	4.x	CSS-first theme; tokens ported 1:1 from DESIGN.md
Maps	Leaflet + react-leaflet + OSM tiles	1.9.4 / 4.2.1	Keyless; five map surfaces; pan/pinch verified
Backend	FastAPI + uvicorn	0.115 / runtime	Stateless; in-memory AppState behind a lock
Routing engine	Custom k-path enumeration + Dijkstra	-	80-path cap; cargo risk-aversion multipliers
External routing	OpenRouteService driving-hgv	API v2	Key server-side only; avoid-polygons on short runs
Android	Kotlin + Compose (BOM 2024.12.01)	2.1.20 / AGP 8.13	Built on Gradle 8.14.3 with JDK 24

Layer	Technology	Version	Assessment
Data target	PostgreSQL + PostGIS	schema only	db/schema.sql mirrors TDD data model; not yet live

5. Screen and Route Inventory

The web application implements fifteen designed screens plus a brand page and an index redirect. All were swept programmatically (React mount + body text length + console error listener).

Route	Screen	Notes
/citizen	Citizen Home - corridor status	OSM map, GPS pin layer, IMD chip, advisories
/citizen/route	Safe Route Planner	ORS-validated geometry, priority pills
/citizen/report	Report a Disruption	EXIF tile, severity tiers, SMS fallback
/citizen/tracking	Live Convoy Tracking	Milestone track, cryo telemetry
/citizen/offline-pass	Offline Transit Pass	Countdown QR, NavIC beacon
/citizen/profile	Profile / settings	Tool links, sign-out removed
/portal	Public Portal	KPIs, corridor health, SMS syntax
/console	Operations Command Center	GIS map, incident feed, freight table
/console/district	District Detail (Pakyong)	Sensor telemetry, contacts
/console/analytics	Analytics & Impact	Charts, severity matrix, fiscal banner
/console/moderation	Report Moderation	Queue table, verification inspector
/console/permits	Digital Transit Permits	Roster, inspector, QR modal
/login /otp /onboarding	Auth flow (bypassed)	Implemented; excluded from default flow
/brand	Emblem / brand asset	Vector emblem, palette swatches

The Android application mirrors the three highest-value citizen screens (corridor status, route planner, report disruption) with the same tokens and mock dataset, and is deliberately login-free.

6. API Surface

Method	Endpoint	Purpose	Behaviour without backend
GET	/api/health	Liveness + model metadata	UI badge switches to SIM MODE
GET	/api/network/status	Segments, alerts, shipments, recs	Embedded engine seeds state
POST	/api/routes/optimize	3-candidate risk-weighted plan	Embedded Dijkstra equivalent
POST	/api/reports	Citizen report ingest + re-score	Local mutation of snapshot
POST	/api/simulate/disruption	Demo event injection	Local injection
POST	/api/simulate/reset	Restore baseline dataset	Local reset

Method	Endpoint	Purpose	Behaviour without backend
POST	/api/shipments/{id}/reroute	Apply allocation suggestion	Local re-plan

All mutating endpoints return the complete snapshot, which keeps the map, alert feed, shipment table and recommendation deck in sync without websocket traffic.

7. Integration Audit

7.1 OpenRouteService (live road routing)

The user-supplied ORS key is loaded from backend/.env (gitignored) via python-dotenv and used exclusively inside the ORS client module; the browser bundle never sees it. The client decorates the engine's fastest candidate with real-road distance, duration and geometry. Three ORS platform limits were discovered and engineered around:

ORS limit	Original failure	Resolution
Avoid-polygon area below 200 km^2	Segment-length bounding box rejected (HTTP 400, code 2003)	Midpoint boxes of ~40 km^2, at most four per request
Avoid-area routes capped at 150 km	Siliguri-Gangtok-class queries rejected (HTTP 400, code 2004)	Avoidance applied only when straight-line distance is 140 km or less
Payload size / render cost	13,360-point geometry stalled the Leaflet layer	Server-side decimation to at most 500 points

Verified result for the planner query: 114 km, 1.5 h, 596-point geometry from Siliguri (26.71 N, 88.43 E) to Gangtok (27.34 N, 88.61 E), flagged orsValidated=true and rendered as the primary corridor.

7.2 Basemap and terrain

Standard OpenStreetMap raster tiles are used keylessly with attribution; the earlier CARTO dark basemap was dropped when it began requiring an API key. Bhuvan (ISRO) WMS is available as an optional overlay on the government heatmap via environment configuration and degrades to no-overlay when unreachable. The ORS key, OSM tiles and Bhuvan WMS are the only external dependencies; all screens remain functional offline against the embedded simulator.

8. Engineering Issues Found and Resolved

Fifteen defects and constraints were identified during development and browser verification. Each was closed with a regression check before delivery.

ID	Area	Issue	Resolution
INF-01	Build	React 19 paired with react-leaflet 4.x peer range produced duplicate map canvases and blank routes	Pinned React 18.3.1 and react-leaflet 4.2.1
INF-02	Runtime	Leaflet ESM build exposes no default export; L.divIcon threw on pin layers	Named imports (divIcon, marker)

ID	Area	Issue	Resolution
INF-03	CSS	Page-enter transform on the phone shell trapped position:fixed, pushing the bottom nav below the viewport	Shell transform removed; content-level animations retained
INF-04	CSS	Map HUD rendered beneath Leaflet panes	isolation:isolate on the map container
UX-01	Layout	Cargo priority pills clipped at card edge	Labels shortened; all three fit at 320 px
UX-02	Runtime	Stale permanent tooltip after planner re-plan	Marker keyed by label state to force remount
APP-01	State	Missing planRoute action crashed the planner to blank	Action added with 2.5 s fail-fast fallback
APP-02	State	useEffect import removed during cleanup, blanking map pages	Import restored; full-route sweep added
INT-01	ORS	Avoid-polygon area above platform cap	Capped midpoint boxes (see section 7.1)
INT-02	ORS	Avoid-area route distance cap	Conditional avoidance by route length
INT-03	ORS	Oversized geometry payload	Server-side decimation to 500 points
INT-04	ORS	Validated result dropped when only one candidate existed	Attached to sole route as fallback
ALG-01	Engine	ETA model over-penalised risky corridors multiplicatively	Linear risk-hours formulation (0.25 h per risk point)
ALG-02	Engine	Three route labels collapsed onto one identical chain	Candidate enumeration with distinct Fastest/Safest/Recommended selection
DATA-01	Graph	Planner story referenced Siliguri but graph had no node	Siliguri gateway node + 92 km NH-10 artery added

9. Verification Evidence

Verification was black-box GUI testing through an in-app browser at mobile (430 px) and desktop (1440 px) viewports, plus API-level probes with curl. Evidence screenshots are retained in gui-test-screenshots/ (p1 through p33).

Check	Method	Result
Route sweep (17 pages)	Programmatic mount + body-text + error listener	17/17 render, zero blank pages
Auth-free entry	Open / then confirm /citizen and 5-tab nav	Pass; nav pinned at viewport bottom
Report to moderation loop	Submit hazard - confirm - inspect console queue	Pass; queue entry + re-score logged
Inject disruption	Command-bar action, ticker and alert check	Pass; segment closed, shipments flagged
Moderation dispatch	Queue action, log inspection	Pass; item consumed, controller log updated
Permit issuance	Emergency pass modal, roster + inspector	Pass; new permit selected automatically
ORS live route	API probe + planner render	Pass; 114 km, 596-pt geometry, orsValidated

Check	Method	Result
APK signature	apksigner verify --print-certs	Pass; V2 debug certificate
Android build	Gradle assembleDebug (JDK 24)	Pass; 15.6 MB APK

10. Security and Privacy Audit

Area	Finding	Assessment
API key handling	ORS key lives in backend/.env (gitignored), read via python-dotenv, used only server-side	Pass for prototype; rotate before any public deployment
Authentication	Auth gate removed for the demonstration; login/OTP screens and RBAC-ready routes remain implemented	Accepted scope cut; must be restored (TDD section 8) before production
APK signing	V2-signed with the debug keystore	Expected for a prototype; create a release keystore before distribution
Personal data	Seed dataset contains mock driver and consignee names only; no live PII	Aligned with DPDP 2023 minimisation for the demo
Transport	All API traffic is localhost HTTP in development; deploy behind NGINX TLS per TDD section 8	Open item for production
Input handling	Report severity bounded server-side (1-5); unknown segment or shipment returns HTTP 404/422	Pass

11. Accessibility and Performance

The web client targets WCAG 2.1 AA: status colours are paired with text labels and indicator dots, touch targets meet 44 px on citizen screens, a skip link and ARIA roles are present, and every animation collapses under prefers-reduced-motion. Contrast was tuned against the DESIGN.md pairings (primary text on white is 98:1; muted text on cards clears 4.5:1). Performance: the production bundle is 384 kB (120 kB gzipped) including Leaflet; route-level code splitting and the PWA offline cache from the TDD remain open items. The embedded fallback engine enumerates at most 80 simple paths and answers plans in under 40 ms on the reference dataset.

12. Known Limitations and Recommendations

#	Limitation	Recommendation
1	All dataset state is simulated and held in process memory	Swap AppState for PostGIS + Redis per docker-compose; schema is ready
2	Risk model is a transparent stand-in formula, not a trained model	Train XGBoost on IMD/Bhuvan/NDMA history behind the same interface (engine/risk.py)
3	Authentication bypassed for the demonstration	Restore NIC OTP flow and RBAC before any government exposure
4	ORS free tier quotas (2,000/day) and avoid-area caps	Cache validated routes; queue avoid-area queries by district
5	Single-process backend, no HTTPS in development	NGINX + TLS termination, Gunicorn workers per TDD deployment section

#	Limitation	Recommendation
6	No automated frontend tests	Add Playwright route sweep as CI, matching the manual 17-route audit
7	Android release channel unsigned	Generate a release keystore; add Play in-app updates later

13. Conclusion

NER-Connect delivers a coherent, working demonstration of AI-assisted corridor management for the North Eastern Region: seventeen functioning web routes, a backend that validates plans on the real road network, and a native Android companion - all speaking one design language and one dataset. The engineering audit found fifteen issues during development; all were resolved and re-verified, and the remaining open items are production hardening steps (trained model, live database, authentication, release signing) that are documented with their intended implementations in the Technical Design Document. The prototype is demo-ready and code-complete for the SIH evaluation criteria.

Field	Value
Report title	NER-Connect Technical Audit Report
Scope	Web prototype, FastAPI backend, Android application
Audit date	26 September 2026
Prepared for	MDoNER - Smart India Hackathon evaluation
Verification status	17/17 web routes, backend live, APK V2-signed